

EMBER

Expert Mixture for Behavioral Encoding and Routing

A framework for universal behavioral prediction via dynamic mixture of experts, automatic latent variables and probabilistic error attribution

August 2026 — Conceptual research document

Abstract

EMBER is a behavioral prediction system for interactive agents (players, users, operators) based on a dynamic Mixture of Experts architecture. Each expert (profile) maintains its own latent variable space, discovered automatically through training — no semantic meaning is imposed on any dimension. A routing mechanism based on pure distance computation in the latent space directs each observed agent toward the K nearest profiles, whose outputs are blended proportionally. Profiles are created, merged and evolved autonomously through a lifecycle driven by configurable distance thresholds. A vote-based confirmation system prevents overfitting on noise. Progressive pruning with compressed archives protects against silent model degradation. The system is domain-agnostic: only the input extraction function is domain-specific; all other components are universal.

1. System Overview and Global Architecture

Ember addresses the problem of modeling human behavioral patterns in interactive domains (games, instruments, interfaces) without imposing any semantic structure on the internal representations. The key insight is that a single neural network cannot efficiently capture the full diversity of human behavioral modes — a player who is stressed, focused or fatigued requires fundamentally different prediction mechanisms. Ember resolves this through a **dynamic Mixture of Experts (MoE)** where each expert (profile) specializes on a behavioral cluster.

The architecture rests on five principal components:

1. A **single dual-mode neural network** that either encodes context into a latent vector z , or updates profiles from system outputs
2. A **pure distance-based router** that selects the K nearest profiles in z -space — no learned network involved
3. An **autonomous profile lifecycle** driven by configurable distance thresholds
4. A **vote-based confirmation system** that prevents error overfitting
5. A **progressive pruning pipeline** with compressed archives for model safety

What Ember is NOT:

- A perfect clone — reproducing every action is impossible without full knowledge of the player's mental state
 - An emotion detector — Ember never labels states as "stress" or "rage"; it identifies statistical correlations
 - A system with predefined variable semantics — like an LLM, z -dimensions have no imposed meaning
 - A boolean system — everything is probabilistic; not "this is Profile A" but "70% Profile A, 30% Profile B"
-

2. Input Representation

2.1 Raw game data

The primary input to Ember is a **raw vector of domain-specific game state variables** at the moment of each player action. The content and dimensionality of this vector are entirely determined by the domain implementor — Ember imposes no constraints.

Examples for Geometry Dash:

- Obstacle positions relative to the player
- Level completion percentage
- Consecutive death count
- Distance to personal best
- Number of deaths at this specific checkpoint

These variables are provided during the supervised training phase and are **progressively pruned** as the model internalizes them into its latent representations. Only the input extraction function is domain-specific; all other Ember components are universal across domains.

2.2 Fundamental deviations δt , δp , δa

In addition to raw game data, Ember computes three **fundamental deviation scalars** at each action, representing the discrepancy between what the player did and what was expected.

These three components are universal across all domains:

$(\delta t, \delta p, \delta a)$ = (temporal deviation, positional deviation, action deviation)

Component	Definition	Always applicable
δt	Signed temporal offset between actual and expected action timing	Yes
δp	Spatial/directional distance from optimal position or target	~
δa	Degree to which the chosen action matches the expected action type	~

The implementor may extend these three fundamental deviations with additional domain-specific components. The three fundamentals serve as a universal baseline that enables cross-domain comparison of behavioral patterns.

Domain	δt	δp	δa
Geometry Dash	ms from sliding average	—	jumped early/ jumped late
osu!	ms from hit window	px from circle center	correct circle / miss
FPS	ms reaction time	px from target	correct target / wrong
Instrument	ms from score timing	semitones from target note	correct note / wrong

2.3 Reference computation and cold start

Computing deviations requires a reference — the expected action. This creates a bootstrapping dependency: the model cannot predict before training, yet deviations require predictions. Ember resolves this through a **two-phase reference scheme**:

Phase 1 : reference = sliding average of the player's own recent actions

Phase 2 : reference = model prediction (once sufficient training has occurred)

The transition from Phase 1 to Phase 2 is gradual and automatic. Deviations computed from a sliding average are noisier but **immediately available**, allowing the model to begin training from the very first session.

3. The Latent Space z — Automatic Variables

3.1 Analogy with LLM embeddings

In large language models, no engineer defines that dimension 512 of an embedding represents "linguistic register formality". This meaning emerges as a consequence of the training objective — the model is forced to encode in its vectors exactly what is necessary to predict the next token. **Ember applies this principle directly.**

Each profile maintains its own **independent latent space** $\mathbf{z}$ of configurable dimensionality d_z (default: 64). No dimension has a predefined meaning. The semantic content of each dimension emerges from what is predictively useful for the behavioral cluster that profile specializes on.

Important: The latent space $\mathbf{z}$ is not shared between profiles. Each profile has its own encoder, its own $\mathbf{z}$, and its own decoder. This is what makes each profile a *model in its own right*.

3.2 Composite training objective

The richness of $\mathbf{z}$ is directly proportional to the richness of the training objective. Ember uses three simultaneous prediction heads:

$$L_{\text{total}} = \lambda_1 \cdot L_{\text{action}} + \lambda_2 \cdot L_{\text{timing}} + \lambda_3 \cdot L_{\text{failure}} + \beta \cdot \text{KL}(q_{\Phi} \parallel p)$$

Loss term	Definition	Default weight
L_{action}	Cross-entropy: most probable next action	$\lambda_1 = 1.0$
L_{timing}	NLL of Gaussian: deviation distribution $N(\delta; \mu, \sigma)$	$\lambda_2 = 0.5$
L_{failure}	Binary cross-entropy: probability of action failure	$\lambda_3 = 0.3$
$\beta \cdot \text{KL}$	Regularization of $\mathbf{z}$ toward $N(0, I)$	$\beta = 0.05$

3.3 Mathematical formalization (ELBO)

The training objective is the **Evidence Lower Bound (ELBO)** from variational autoencoders (Kingma & Welling, 2013):

$$L(\Theta, \Phi) = E_{\{q_{\Phi}(\mathbf{z}|\text{hist})\}}[\log P_{\Theta}(a_t | c_t, \mathbf{z})] - \beta \cdot \text{KL}(q_{\Phi}(\mathbf{z}|\text{hist}) \parallel N(0, I))$$

The latent vector $\mathbf{z}$ is sampled via the **reparametrization trick**, maintaining end-to-end differentiability:

$$\mathbf{z} = \mu(\text{hist}) + \sigma(\text{hist}) \odot \epsilon, \quad \epsilon \sim N(0, I)$$

4. The Single Dual-Mode Network

A naive implementation might use two separate networks: one to encode game context into $\mathbf{z}$, and another to update profiles from system outputs. However, this creates a **fundamental coherence problem**: a profile-update network that has never seen the context encoding process cannot update profiles in a way that is consistent with how $\mathbf{z}$ was constructed.

Ember resolves this with a **single network operating in two modes**, controlled by a binary flag.

4.1 Mode 0 — Context encoding (flag = 0)

The network receives raw game data and computed deviations (δt , δp , δa) as input and produces a latent vector $\mathbf{z}_{\text{player}}$ representing the current behavioral position of the player in latent space. This $\mathbf{z}_{\text{player}}$ is then passed to the router.

4.2 Mode 1 — Profile update (flag = 1)

The network receives the system's own outputs as input: current predictions, observed deviations, active profile weights from the router, and vote confirmation signals. It uses this information to **update the weights of the nearest profile**.

Because the same network performed the context encoding in Mode 0, it has an internal representation that is coherent with both the input space and the profile space — enabling meaningful profile updates.

4.3 Why a single network

Key insight: The binary flag is strictly 0 or 1 — never mixed. Mixing raw game data with system outputs in a single forward pass would be semantically incoherent. The flag selects one of two completely separate data flows through the same weight matrix. **Shared weights ensure representational coherence between the two modes.**

```

class EMBERNetwork(nn.Module):

    """Single dual-mode network for Ember."""
    def __init__(self, input_dim_mode0, input_dim_mode1, z_dim=64, hidden=128):
        super().__init__()
        # Separate projection layers for each mode
        self.proj0 = nn.Linear(input_dim_mode0, hidden) # Mode 0
        self.proj1 = nn.Linear(input_dim_mode1, hidden) # Mode 1
        # Shared GRU backbone — coherence between modes
        self.gru = nn.GRU(hidden, hidden, num_layers=2, batch_first=True)
        # Mode 0 heads: produce z
        self.mu_head = nn.Linear(hidden, z_dim)
        self.logvar_head = nn.Linear(hidden, z_dim)
        # Mode 1 head: produce profile update delta
        self.update_head = nn.Linear(hidden, z_dim)
    def forward(self, x, mode: int):
        # mode=0 : x = raw game data + computed deviations
        # mode=1 : x = system outputs (predictions, active weights, observed deviations)
        h = F.silu(self.proj0(x) if mode == 0 else self.proj1(x))
        out, _ = self.gru(h)
        h_last = out[:, -1, :]
        if mode == 0:
            mu = self.mu_head(h_last)
            logvar = self.logvar_head(h_last)
            z = mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)
            return z, mu, logvar
        else: # mode == 1
            return self.update_head(h_last) # delta to apply to nearest profile

```

5. Routing and Profile Mixture

5.1 Distance-based routing (no network)

The router is **not a neural network**. It is a pure mathematical function that computes the Euclidean distance between **z_{player}** and each profile's centroid in latent space, then selects the K nearest profiles:

$$\text{dist}_i = \|z_{\text{player}} - z_{\text{profile}_i}\|_2 \quad \text{for each profile } i$$

$$\text{Top-K} = \{ i : \text{dist}_i \leq \text{K-th smallest distance} \}$$

This design is intentional. The latent space z already encodes all relevant behavioral context — the player's position in z -space **is** the context. A learned router would add parameters and training complexity without providing additional information beyond what z already captures.

5.2 Top-K selection and weighted fusion

Profile weights are computed via softmax over negative distances, with a temperature parameter T controlling the hardness of the routing:

$$w_i = \exp(-\text{dist}_i / T) / \sum \exp(-\text{dist}_j / T)$$

$$\text{output_final} = \sum_i w_i \cdot \text{output_profile}_i$$

- $T \rightarrow 0$: nearest profile dominates (hard routing)
- $T \rightarrow \infty$: uniform mixture of all K profiles (soft routing)
- **Recommended default:** $T = 0.5$

```
def route(z_player, profiles, K=3, T=0.5):
    """Pure distance-based routing. No network required."""
    distances = {pid: np.linalg.norm(z_player - p.centroid)
                  for pid, p in profiles.items()}
    # Select Top-K nearest
    top_k = sorted(distances, key=distances.get)[:K]
    # Softmax weights
    raw = {pid: np.exp(-distances[pid] / T) for pid in top_k}
    total = sum(raw.values())
    weights = {pid: w / total for pid, w in raw.items()}
    return weights

def fuse(weights, outputs):
    """Weighted fusion of profile outputs."""
    return sum(w * outputs[pid] for pid, w in weights.items())
```

6. Profile Lifecycle

6.1 Profile fusion

When the distance between two profile centroids falls below `fusion_threshold`, the profiles are **merged** into a single profile whose centroid is the weighted average of the two, weighted by the number of players associated with each:

$$z_{\text{fused}} = (n_A \cdot z_A + n_B \cdot z_B) / (n_A + n_B)$$

All players associated with the merged profiles migrate automatically to the new fused profile.

6.2 Profile creation

When a player's `z_player` exceeds `creation_threshold` distance from all existing profiles, **and** BOCPD (Bayesian Online Changepoint Detection) simultaneously confirms a regime change, a new profile is created.

The new profile is **initialized as a copy of the nearest existing profile** and subsequently diverges through Mode 1 updates specific to this player. The double condition (distance AND BOCPD) is essential to prevent spurious profile creation from transient noise.

```
def maybe_create_profile(z_player, profiles, bocpd, thresholds):
    distances = {pid: np.linalg.norm(z_player - p.centroid)
                 for pid, p in profiles.items()}
    min_dist = min(distances.values())
    nearest_id = min(distances, key=distances.get)
    changepoint_score = bocpd.update(z_player)
    if (min_dist > thresholds['creation']
        and changepoint_score > thresholds['bocpd_confirm']):
        # Copy nearest profile as starting point
        new_profile = profiles[nearest_id].copy()
        new_profile.id = generate_id()
        profiles[new_profile.id] = new_profile
        return new_profile.id
    return None
```

6.3 Cold start initialization

Ember cannot start with zero profiles — the Top-K router would have nothing to route to. A **mandatory bootstrap phase** is required before any deployment:

1. Collect sessions from N real players (minimum ~50)
2. Train a generic encoder on all collected data
3. Project all N players into z-space
4. K-means clustering → initial profiles (centroids)

After bootstrap, **all profiles are equal in status**. There is no permanent "central profile" — the concept of a central profile refers only to the router mechanism (distance computation), not to any special profile entity. Profiles evolve, merge and are created dynamically from the bootstrap state onward.

7. Error Confirmation and Causal Correlation

7.1 The overfitting-on-noise problem

If every observed error were immediately integrated into a profile, the model would overfit to **irreducible noise** — errors caused by external distractions (chat messages, room interruptions) that have no causal relationship to the game state. The profile would "invent" internal causes for these errors and begin generating spurious errors in similar future contexts.

Fundamental rule: An error is never integrated into a profile until it has been statistically confirmed as repeatable across multiple observations in similar contexts. A single isolated error only increases the residual noise term σ slightly.

7.2 Fuzzy-context vote system

Rather than searching for an exact context match (which may never recur), Ember searches for **similar contexts within a configurable radius** and aggregates their votes weighted by proximity:

$$\text{vote_total} = \sum_i \text{vote_i} \cdot \exp(-\text{dist}(\text{C_current}, \text{C_i}) / \text{radius})$$

if $\text{vote_total} \geq \text{confirmation_threshold}$ → confirmed, integrated into profile
if $\text{vote_total} < \text{confirmation_threshold}$ → pending, $\sigma \ += \ \epsilon$

7.3 Probabilistic cause-deviation correlations

Instead of attempting to detect and name player emotional states (values that are arbitrary and player-dependent), Ember identifies **statistical correlations** between contextual events and behavioral deviations:

$$P(\text{deviation } D \mid \text{event } E) = \text{confirmed_votes}(E \rightarrow D) / \text{total_observations}(E)$$

These correlations emerge naturally from the vote system. Ember never labels a state as "stress" or "rage" — it reports that a specific event causes a specific deviation pattern with a specific probability. Emotional semantics may be inferred by the end user from these correlations, but are never imposed by the system.

8. Progressive Pruning and Anti-Stack Archives

As Ember trains, some input variables become redundant because their information has been internalized into `z`. Progressive pruning identifies and removes these variables to reduce computational cost and improve generalization.

Pruning is triggered automatically when prediction accuracy exceeds `pruning_accuracy_threshold`. The pipeline:

1. Tests the model without variable X on validation data
2. If degradation is **strong**: trains a copy without X for a few thousand iterations. If it catches up → replace, archive original
3. If degradation is **weak or none**: tests on other samples (different players, different sessions). If consistently stable → replace, archive original
4. A compressed snapshot is archived **before every replacement**

A **stack** is a silent accumulation of small model degradations that renders the model unusable without an obvious single failure point. To protect against stacks, a compressed snapshot is archived before every replacement. If anomalous behavior is detected at any future point, the system can roll back to the most recent valid archive.

9. Error Taxonomy

All observed errors belong to one of three categories. Only the first two are modelable:

Type	Temporal signature	Ember treatment	Modelable
Material	Constant bias, stable variance, same context → same error	Integrated as systematic bias in profile	Yes
Behavioral	Correlated with contextual event, repeatable across samples	Vote confirmation → probabilistic correlation → profile adjusted	Yes
Irreducible residual	Isolated spike, no correlation, cause outside feature space	σ increased slightly, never integrated into profile	No

Hard limit — not circumventable without external sensors: A chat message, a person speaking in the room, a sudden thought — these causes do not exist in the game feature space. No quantity of parameters or architectural complexity can recover information that was never observed. This is an **information-theoretic limit** in the sense of Shannon (1948), not a computational one.

10. Application to Geometry Dash

Input vector (Mode 0)

Variable	Computation	Proxy for
Obstacle distances	[dist to obj 1, dist to obj 2, ...]	Immediate danger layout
Level percentage	current_pos / total_length	Pressure and familiarity
Deaths at this point	historical death count at this %	Local difficulty
Distance to PB	current_pct – personal_best_pct	Stress / novelty

Variable	Computation	Proxy for
Session duration	time played continuously (min)	Fatigue proxy
δt	actual_jump_time – sliding_avg (ms)	Temporal deviation
δp	actual_position – expected_position	Positional deviation
δa	0 if jumped correctly, ± 1 otherwise	Action deviation

Jump success probability

Given the active profile blend's predicted deviation distribution, the probability of a successful jump is computed analytically:

$$P(\text{success}) = P(|\delta t_{\text{predicted}}| < \text{tolerance_obstacle})$$

$$= \text{erf}(\text{tolerance_obstacle} / (\sigma_{\delta t} \cdot \sqrt{2}))$$

Where **tolerance_obstacle** depends on the obstacle type (standard block > orb > spike) and $\sigma_{\delta t}$ is the predicted timing variance from the active profile blend for this context.

11. Multi-Domain Generalization

The strict separation between the domain-specific input extraction function and the universal Ember core enables deployment across arbitrary interactive domains. The implementor provides a **single function**:

```
def extract(game_state: dict) -> np.ndarray:
    # Only this function is domain-specific.
    # Returns a numpy vector of any length.
    # Ember auto-adapts its projection layer to the vector size on first call.
    ...
```

On cross-installation compatibility

Two Ember installations with different input extraction functions are **fundamentally incompatible** — their latent spaces are built on different observations and cannot be merged. This is expected behavior, not a limitation to be solved.

Users may share:

- Input extraction functions for a given domain
- Configuration files (thresholds, K, T, etc.)
- Performance benchmarks

Users **cannot** share trained profiles across installations with different extraction functions.

Transfer rule: **z** encodes what would be true of the agent even if the domain changed (reaction latency, risk tolerance, fatigue patterns). The extraction function encodes what is specific to this domain (obstacle types, timing windows).

12. Configurable Parameters

Parameter	Default	Effect if increased	Effect if decreased
K (Top-K)	3	Smoother blending, more compute	Less profile diversity
fusion_threshold	0.3	Fewer fusions, profiles proliferate	Too many fusions
creation_threshold	0.8	Fewer new profiles created	Too many profiles
T (temperature)	0.5	Uniform mixture	Nearest profile dominates
z_dim	64	Richer z, overfitting risk	Under-expressive

Parameter	Default	Effect if increased	Effect if decreased
beta (KL weight)	0.05	z collapses toward $N(0, I)$	z unregularized
vote_radius	0.2	Fast confirmation, less precise	Slow, precise
confirmation_threshold	3 votes	Hard to confirm	Overfits on noise
pruning_accuracy_threshold	0.85	Late pruning	Premature pruning
bocpd_hazard_rate	0.02	More changepoints detected	Fewer detected

13. References

- [1] Kingma & Welling (2013). *Auto-Encoding Variational Bayes*. ICLR 2014.
- [2] Adams & MacKay (2007). *Bayesian Online Changepoint Detection*. arXiv:0710.3742.
- [3] Jacobs et al. (1991). *Adaptive Mixtures of Local Experts*. Neural Computation 3(1).
- [4] Shazeer et al. (2017). *Outrageously Large Neural Networks: Sparse MoE*. ICLR.
- [5] Rakelly et al. (2019). *PEARL: Efficient Off-Policy Meta-RL*. ICML.
- [6] Higgins et al. (2017). *beta-VAE: Learning basic visual concepts*. ICLR.
- [7] Alos-Ferrer & Garagnani (2023). *HMM for studying behavioral types*. Frontiers in Behavioral Economics.
- [8] Shannon (1948). *A Mathematical Theory of Communication*. Bell System Technical Journal.
- [9] Zhu et al. (2025). *SWIRL: Switching IRL with history dependency*. ICML.

Made by Thetecker.

If you have questions contact me at : contact.thetecker@gmail.com =3